



AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

**Dept. of Computer Science
Faculty of Science and Technology**

CSC 2210: OBJECT ORIENTED PROGRAMMING 2

Fall 2025-2026

Section: V

Group No: 6

Project Report On Trading Management System

Supervised By

MD. Raihan Talukder

Submitted By:

Name	ID
1. Md Imran Hasan Shuvo	24-56808-1
2. Dina Akter	23-52352-2
3. Md. Arafat Al Shahriar	24-56816-1
4. Monia Islam Mukti	24-55933-1

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	Evaluation Definition				Total =
Requirement fulfillment	Properly demonstrate a real-life scenario-based project with proper functional requirement identification for the Object-Oriented Programming project development activities.				
Validation	Ensuring the ability of students' proper demonstration on validation forms in their system in terms of dealing with the data.				
Verification	Identifying if the students can verify the system data along with proper functional requirements in terms of data flow.				

Table of Contents

1. INTRODUCTION	3
2. FEATURE LIST	3,4
3. ER DIAGRAM	5
3.1 Normalization	5,6
3.2 Finalization	6,7,8
4. UML DIAGRAM.....	9
4.1 Use Case Diagram.....	10
4.2 Activity Diagram.	11,12,13
5. CONCLUSION.....	14

1. INTRODUCTION

The **Trading Management System** is a **desktop-based application** designed to simplify and automate the daily operations of a trading business. Developed using **C# Windows Forms** and connected to a **SQL Server database**, the system ensures secure and efficient management of users, customers, stock, and financial transactions.

The system features a **secure login module** with password hashing using **BCrypt**, and supports multiple user roles including **Owner, Manager, Cashier, Salesman, and Stock Manager**, each with specific access permissions. New users can sign up using **OTP-based email verification**, ensuring authenticity and security during registration.

For customers, the system allows **account creation, status activation/deactivation**, and management of contact details. It maintains **transaction histories**, records **partial payments**, calculates **due balances**, and displays information in real-time for easy monitoring.

The **stock management module** enables users to **add, update, and track products** efficiently. It supports **weighted average pricing**, prevents duplicate entries, and allows addition of new products dynamically. All operations are performed with **data integrity and transactional safety** to prevent errors.

Overall, the project provides a **complete, secure, and user-friendly solution** for managing business operations, reducing manual work, and improving accuracy and efficiency in a trading environment.

2. FEATURE LIST

Owner (Full Control)

- **Prompt Name:** user1
- **Description:** Full access to the system.
- **Permissions / Actions:**
 - **Manage user accounts** (create, edit, deactivate)
 - **Manage customer accounts** (create, update, delete)
 - **View, add, or update stock**
 - **Process payments and manage customer balances**
 - **Generate invoices**
 - **Access all reports and dashboards**
 - **Inherits all Manager-level permissions**
- **Special Notes:** Can override other roles' actions

Manager (Daily Operations)

- **Prompt Name:** user2
- **Description:** Handles daily operations and business supervision.
- **Permissions / Actions:**
 - **Generate and manage invoices**
 - **Supervise stock levels** (view/update)
 - **Monitor customer accounts and payments**

- Access operational reports
- **Limitations:** Cannot manage user accounts
- **Inheritance:** All Manager permissions are available to the Owner

Salesman (Sales Operations)

- **Prompt Name:** user3
- **Description:** Performs sales-related tasks only.
- **Permissions / Actions:**
 - Generate and record **invoices**
 - View **customer details** (read-only)
- **Limitations:** Cannot access **stock updates**, **payments**, or **account management**

Stock Manager (Inventory Focus)

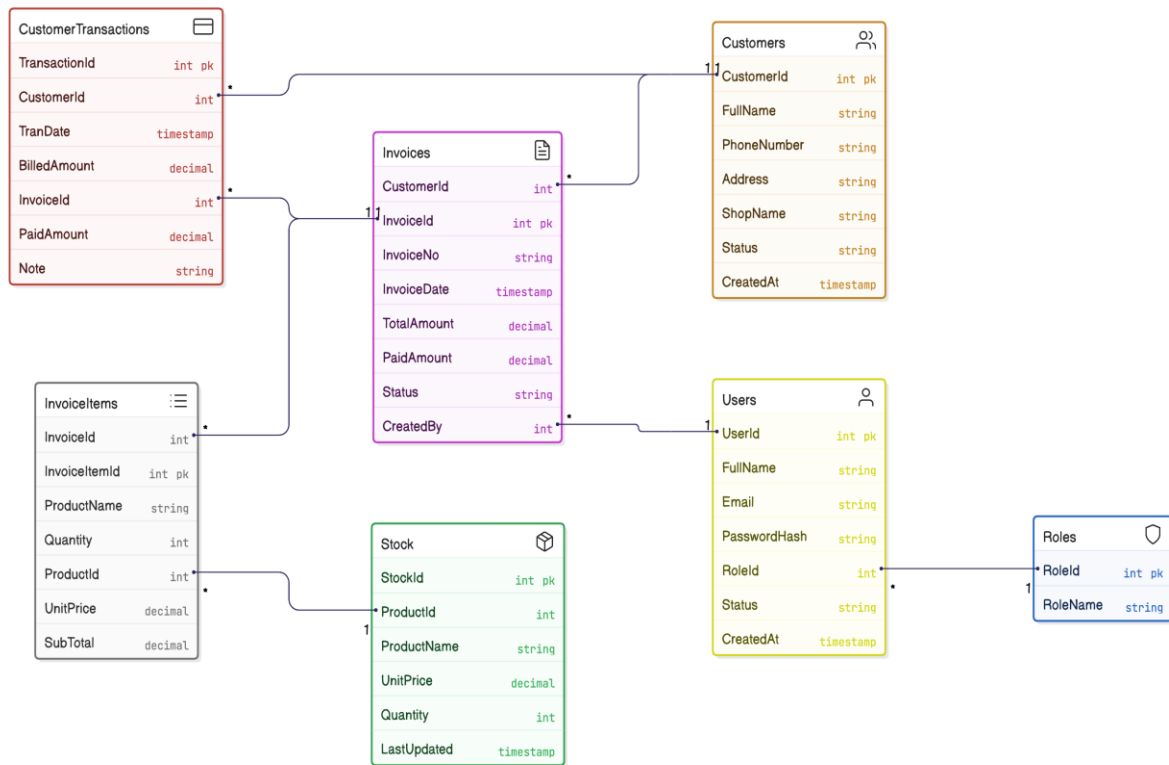
- **Prompt Name:** user4
- **Description:** Focused on inventory management.
- **Permissions / Actions:**
 - Add or update **stock entries**
 - Calculate **weighted average prices**
 - Refresh and maintain **stock grids**
- **Limitations:** Cannot generate invoices or manage customers

Cashier (Payment Handling)

- **Prompt Name:** user5
- **Description:** Handles customer payments and transaction recording.
- **Permissions / Actions:**
 - View **customer transactions**
 - Deposit **partial payments**
 - Refresh customer **due balances**
- **Limitations:** Cannot manage **stock**, **generate invoices**, or **manage users**

3. ER DIAGRAM

Trading Management System



The ER diagram represents a Tutor Management System that manages users, tutors, students, courses, enrollments, and payments. The system is designed using a relational database structure with primary keys (PK) and foreign keys (FK).

3.1 Normalization

First Normal Form (1NF)

No Change.

Second Normal Form (2NF)

- Partial dependency: 'RoleName' depends on 'RoleId'
- Partial dependency: 'RoleName' depends on 'RoleId'
- Partial dependency: 'ProductName' depends on 'ProductId'
- Partial dependency: 'ProductName' depends on 'ProductId'
- Partial dependency: 'InvoiceNo' depends on 'InvoiceId'
- Partial dependency: 'InvoiceNo' depends on 'InvoiceId'
- Partial dependency: 'InvoiceNo' depends on 'InvoiceId'
- Partial dependency: 'InvoiceDate' depends on 'InvoiceId'
- Partial dependency: 'InvoiceDate' depends on 'InvoiceId'

- Partial dependency: 'InvoiceDate' depends on 'InvoiceId'
- Partial dependency: 'ProductName' depends on 'ProductId'
- Partial dependency: 'ProductName' depends on 'ProductId'

Recommendations: 1. Move 'RoleName' to table keyed by 'RoleId'

2. Moved 'RoleName' to table keyed by 'RoleId'
3. Moved 'ProductName' to table keyed by 'ProductId'
4. Moved 'ProductName' to table keyed by 'ProductId'
5. Moved 'InvoiceNo' to table keyed by 'InvoiceId'
6. Moved 'InvoiceNo' to table keyed by 'InvoiceId'
7. Moved 'InvoiceNo' to table keyed by 'InvoiceId'
8. Moved 'InvoiceDate' to table keyed by 'InvoiceId'
9. Moved 'InvoiceDate' to table keyed by 'InvoiceId'
10. Moved 'InvoiceDate' to table keyed by 'InvoiceId'
1. Moved 'ProductName' to table keyed by 'ProductId'
12. Moved 'ProductName' to table keyed by 'ProductId'

Issues Found (3):

1. Transitive dependency: Name → Email
2. Transitive dependency: Name → Phone
3. Transitive dependency: Name → Address

Finalization

Tables:

1. Roles

RoleId (PK)

RoleName

Purpose:

Stores role-related information separately to remove partial dependency.

2. Users

UserId (PK)

FullName

Email

PasswordHash

RoleId (FK)

Status

CreatedAt

Foreign Key:

RoleId → Roles(RoleId)

Purpose:
Stores user authentication and role assignment details.

3. Products

ProductId (PK)
ProductName
UnitPrice
Status
LastUpdated

Purpose:
Stores product-related data independently.

4. Stock

StockId (PK)
ProductId (FK)
Quantity
LastUpdated

Foreign Key:
ProductId → Products(ProductId)

Purpose:
Manages inventory quantities for products.

5. Customers

CustomerId (PK)
FullName
PhoneNumber
Address
ShopName
Status
CreatedAt

Purpose:
Stores customer identity and contact information.

6. Invoice

InvoiceId (PK)
InvoiceNo
CustomerId (FK)
CreatedBy (FK)
InvoiceDate
TotalAmount

PaidAmount
Status

Foreign Keys:

CustomerId → Customers(CustomerId)

CreatedBy → Users(UserId)

Purpose:

Stores invoice header information.

7. InvoiceItems

InvoiceItemId (PK)

InvoiceId (FK)

ProductId (FK)

Quantity

UnitPrice

SubTotal

Foreign Keys:

InvoiceId → Invoices(InvoiceId)

ProductId → Products(ProductId)

Purpose:

Stores line-item details for each invoice.

8. Transactions

TransactionId (PK)

InvoiceId (FK)

CustomerId (FK)

TranDate

BilledAmount

PaidAmount

Note

Foreign Keys:

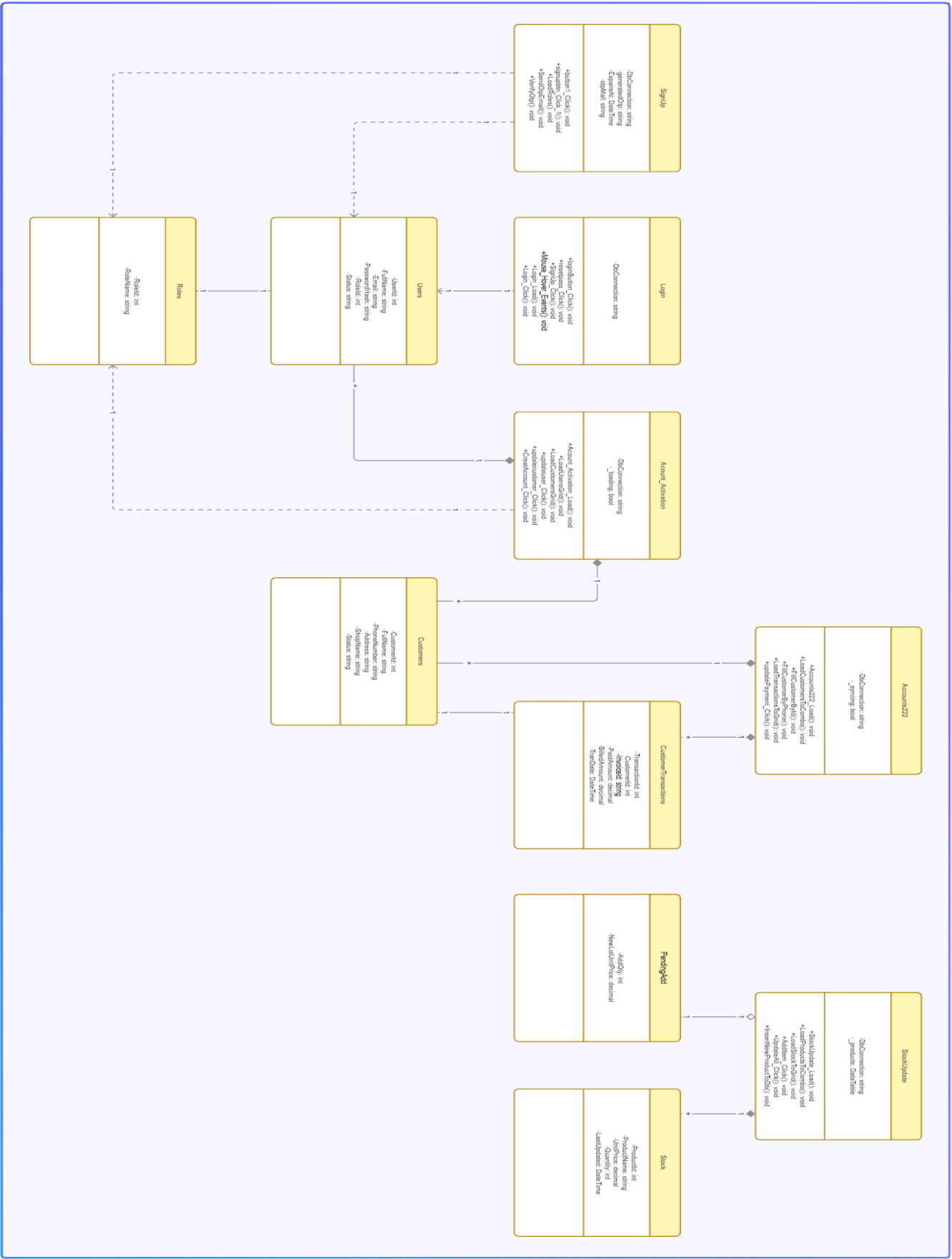
InvoiceId → Invoices(InvoiceId)

CustomerId → Customers(CustomerId)

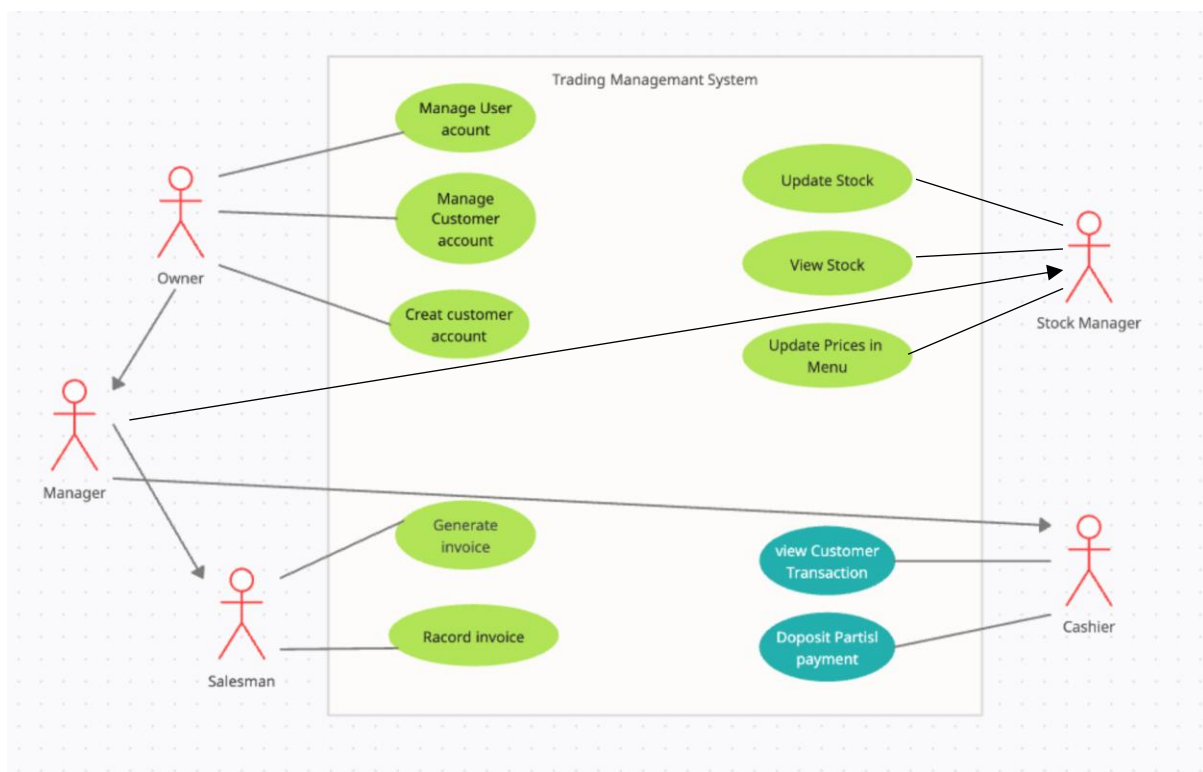
Purpose:

Tracks payment and transaction history.

4. UML DIAGRAM



4.1 Use Case Diagram



Description

In this diagram, the **Trading Management System** is shown as a central system where all operations are performed. The large box represents the system, and the people outside the box represent the different roles that interact with it. Each role is connected to only those features that are permitted for that role.

Full control of the system is assigned to the **Owner**. User accounts are managed, customer accounts are created, and existing customer records are maintained by this role. Through the inheritance arrow, all Manager-level permissions are also granted to the Owner. In this way, every function available in the system can be accessed by the Owner.

Daily operational control is given to the **Manager**. Invoices are generated and business activities are supervised through this role. Manager-level capabilities are inherited by the Owner, ensuring higher authority.

Sales-related tasks are performed by the **Salesman**. Invoices are generated and recorded by this role. Access to stock, payments, or account management is not provided.

Inventory operations are handled by the **Stock Manager**. Stock levels are viewed, quantities are updated, and product prices are modified. No access to sales or payment functions is granted.

Financial transactions are managed by the **Cashier**. Customer transactions are viewed and partial payments are recorded. Stock and customer account management are not accessible from this role.

Each connecting line indicates that a specific action is permitted for a specific role. In this way, responsibility is clearly separated, system security is maintained, and an organized workflow is ensured.

4.2 Activity Diagram.

Description

The activity diagram illustrates the flow of operations in the **MS Malek Traders** desktop-based trader management system. It models key user interactions, including **Login, SignUp, Stock Management, and Customer Account Management**. The diagram captures validation checks, decision points (like OTP verification and confirmation dialogs), iterative processes (such as loading multiple transactions or updating multiple stock items), and parallel activities (grid updates vs database updates). This visualization helps developers and stakeholders understand the system behavior, user navigation, and data processing, making it suitable for a software design document.

1. Activity Diagram (Textual Representation / Guide)

2. Start

- **Start Node → Login or SignUp Choice**

3. Login Flow

4. **Action:** Enter Email & Password

5. **Decision:** Are fields valid?

- **No** → Show error → **End**
- **Yes** → Continue

6. **Action:** Check account in database

7. **Decision:** Are credentials valid & account active?

- **No** → Show error → **End**
- **Yes** → Navigate to **Dashboard** → **End**

8. SignUp Flow

1. **Action:** Enter Name, Email, Password, Role

2. **Decision:** Validate Email & Password

- **No** → Show error → **End**
- **Yes** → Continue

3. **Action:** Send OTP to email

4. **Decision:** OTP verified?

- **No** → Retry OTP → **Loop**
- **Yes** → Create user with 'inactive' status

5. **Action:** Show success message
 6. **Action:** Return to Login → **End**
-

9. Stock Management Flow

1. **Action:** Select/Type Product → Enter Quantity & Unit Price
 2. **Decision:** Does product exist?
 - **Yes** → Add pending stock to grid
 - **No** → Confirm addition → Insert into database → Add to grid
 3. **Action:** Update All (Loop for multiple stock rows)
 - Calculate weighted average price
 - Update database
 - Refresh grid → **End**
-

10. Customer Accounts Flow

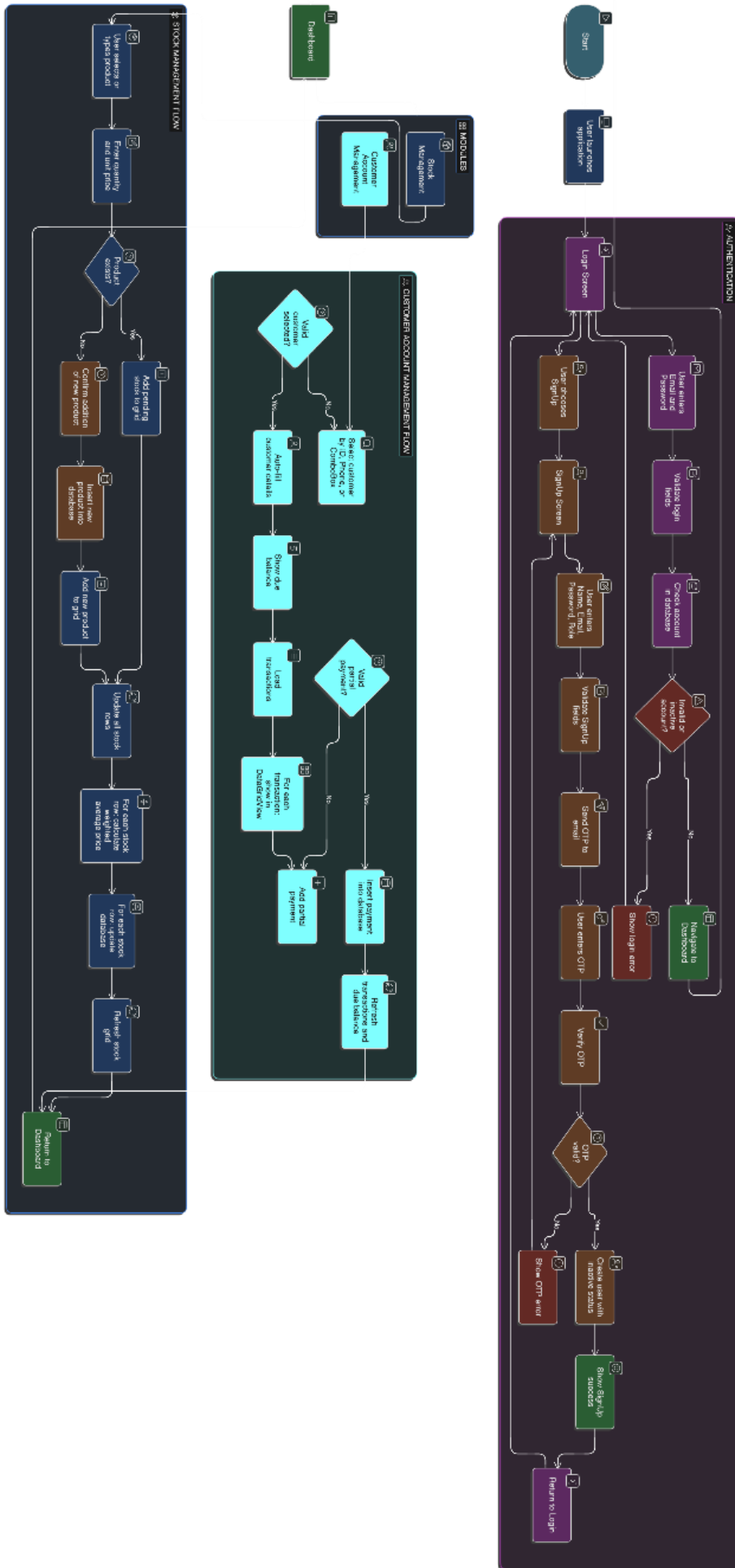
1. **Action:** Select Customer (ID, Phone, ComboBox)
 2. **Action:** Auto-fill details → Show due balance
 3. **Action:** Load transactions (Loop for multiple records) → Show in DataGridView
 4. **Action:** Add partial payment → Insert into database
 5. **Action:** Refresh transactions & due balance → **End**
-

11. Parallel Flows

- Stock updates in grid **and** database updates can happen in parallel for efficiency.
 - Loading customer transactions and displaying due balance can happen simultaneously.
-

12. End

- **Dashboard** (for Login/Stock updates)
- **Success message** (for SignUp/Customer payment updates)



5. CONCLUSION

The **Trading Management System** successfully integrates user management, customer account handling, stock updates, and financial transaction tracking into a single, secure, and efficient desktop application. By automating key business operations and providing role-based access, the system reduces manual effort, minimizes errors, and ensures real-time data accuracy. Its modular design, combined with security features such as password hashing and OTP verification, makes it a reliable solution for managing a trading business effectively. Overall, the project demonstrates how technology can streamline business processes and enhance operational efficiency.